SIAM J. SCI. COMPUT.
Vol. 38, No. 1, pp. A545–A566
© 2016 Society for Industrial and Applied Mathematics

A HYBRIDIZABLE DISCONTINUOUS GALERKIN METHOD FOR THE p-LAPLACIAN*

BERNARDO COCKBURN† AND JIGUANG SHEN†

Abstract. We propose the first hybridizable discontinuous Galerkin method for the p-Laplacian equation. When using polynomials of degree $k \geq 0$ for the approximation spaces of u, $\boldsymbol{\nabla} u$, and $|\boldsymbol{\nabla} u|^{p-2}\boldsymbol{\nabla} u$, the method exhibits optimal $k+1$ order of convergence for all variables in L^1- and L^p-norms in our numerical experiments. For $k \geq 1$, an elementwise computation allows us to obtain a new approximation u_h^* that converges to u with order $k+2$. We rewrite the scheme as discrete minimization problems in order to solve them with nonlinear minimization algorithms. The unknown of the first problem is the approximation of u on the *skeleton* of the mesh but requires solving nonlinear local problems. The second problem has the approximation on the elements as an additional unknown but it only requires solving linear local problems. We present numerical results displaying the convergence properties of the methods, demonstrating the utility of using frozen-coefficient preconditioners, and indicating that the second method is superior to the first one even though it has more unknowns.

Key words. hybridization, discontinuous Galerkin methods, degenerate elliptic equations, p-Laplacian operator, nonlinear minimization algorithms, preconditioning, postprocessing

AMS subject classifications. 65N30, 65M60, 35J60, 35J70

DOI. 10.1137/15M1008014

1. Introduction.

In this paper, we develop the hybridizable discontinuous Galerkin (HDG) method for the so-called p-Laplacian equation with $1 < p < \infty$, in mixed form:

$$\boldsymbol{q} + \boldsymbol{\nabla} u = 0, \ \boldsymbol{\sigma} - |\boldsymbol{q}|^{p-2}\boldsymbol{q} = 0, \ \ \nabla \cdot \boldsymbol{\sigma} = f \ \text{ in } \Omega; \ u = g \ \text{ on } \partial\Omega. \tag{1.1}$$

Here, $\Omega \subset \mathbb{R}^2$ is a two-dimensional polyhedral domain. The variable $\boldsymbol{\sigma}$ is considered to the *flux* associated to u. We assume that $f \in L^2(\Omega)$, $g \in W^{1,1/p^*}(\partial\Omega)$, where $1/p + 1/p^* = 1$, to ensure that the problem (1.1) has a unique solution $u \in W^{1,p}(\Omega)$; see [3, 20].

The p-Laplacian operator appears in various applications of practical interest. In fluid mechanics, it is appears as the model for non-Newtonian fluids obeying the power law or the Carreau law; see, for example, [22, 27, 15]. In particular, when $1 < p < 2$, this models the *shear-thinning* behavior of fluid, which is common in practice. The p-Laplacian operator also appears in the study of flow in porous media [32], where Forchheimer type corrections to the Darcy law give rise to a p-Laplacian operator with $p = 1.5$. In solid mechanics, the p-Laplacian equation (1.1) with $f = 1$ and $g = 0$ models a problem from elastic-plastic torsional creep [25]. Specifically, $p = 2$ describes perfectly elastic behaviors, while $p \to \infty$ describes perfect plastic behaviors. In [29], the p-Laplacian is interpreted from a game-theoretic perspective, where it models the game which is termed *random tug of war with noise*. In particular, large

*Submitted to the journal's Methods and Algorithms for Scientific Computing section February 11, 2015; accepted for publication (in revised form) November 30, 2015; published electronically February 23, 2016.

http://www.siam.org/journals/sisc/38-1/M100801.html

†School of Mathematics, University of Minnesota, Minneapolis, MN 55455 (cockburn@math.umn.edu, shenx179@math.umn.edu). The first author's research was supported in part by the National Science Foundation (grant DMS-1115331) and by the University of Minnesota Supercomputing Institute.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

p is taken into consideration when the noise of random walk is small. Furthermore, many other applications, for example, [6, 30, 28, 5], treat the p-Laplacian operator as a building block for more complicated models.

Due to its nonlinear nature, the numerical approximation of the p-Laplacian operator is much more difficult than that of the linear Laplacian ($p = 2$). One of the major difficulties is that when $\boldsymbol{q}$ is close to zero, the p-Laplacian operator degenerates when $p > 2$ and becomes singular when $1 < p < 2$. Moreover, small or large gradients lead to poorly conditioned global systems and hinder the effort of linearization (as small or large p do too). As pointed out as early as 1975 by Glowinski and Marrocco [20], the *Newton–Raphson iteration* cannot be successfully used for this problem. Minimization algorithms (with or without restrictions) might be more reasonable to use given that (see [3, 9]) the exact solution $u \in W_g^{1,p}(\Omega) := \{u \in W^{1,p}(\Omega) : u = g \ \text{ on } \partial\Omega\}$ is itself the solution of

$$J_\Omega(u) \le J_\Omega(w) \quad \forall w \in W_g^{1,p}(\Omega), \quad \text{where} \quad J_\Omega(w) := \frac{1}{p}\int_\Omega |\boldsymbol{\nabla} w|^p - \int_\Omega f\, w. \tag{1.2}$$

Many finite element methods for the p-Laplacian use this approach, as we next describe.

The standard continuous Galerkin (CG) finite element method for the p-Laplacian equation has been extensively studied. In the pioneering work of Glowinski and Marrocco [20], the CG method is analyzed and is solved by using of an *augmented Lagrangian* method. They conducted experiments on a test problem whose solution is

$$u(x,y) = \frac{p-1}{p}\left(\frac{1}{2}\right)^{\frac{1}{p-1}} (1-(x^2+y^2))^{\frac{p}{2(p-1)}}). \tag{1.3}$$

The results show that their method is able to converge to the kink at the origin by using piecewise-linear polynomials. Almost 20 years later, Barrett and Liu [3] revisited the analysis of the piecewise-linear CG finite element approximation and improved the error estimates. The numerical results confirmed that when the solution is sufficiently smooth, the piecewise-linear CG approximation to u is optimal in $W^{1,p}$-norm. More importantly, for the first time, they proposed to solve the resulting minimization problem by the so-called PR-nonlinear conjugate gradient method. Later, Huang, Li, and Liu [23] developed a preconditioner for nonlinear gradient descent algorithms arising from the CG approximation. Their numerical results demonstrated mesh-independent convergence and robustness on the test problem (1.3) using nonuniform meshes. Recently, Zhou and Feng [33] pointed out that the *line search* procedure is in general rather expensive for the finite element methods. Motivated by the work of Shi and Shen [31], they adopted their step length estimation after conducting only one standard line search in the first iteration. This reduced the CPU time significantly.

Compared to the standard CG finite element method, very limited work has been conducted for the mixed finite element methods or discontinuous Galerkin (DG) methods. Farhloul and Manouzi formulated and analyzed the first mixed method in [19, 18] by discretizing the following mixed formulation using the lowest order Raviart–Thomas finite element:

$$|\boldsymbol{\sigma}|^{p^*-2}\boldsymbol{\sigma} + \boldsymbol{\nabla} u = 0, \quad \nabla\cdot\,\boldsymbol{\sigma} = f \ \text{ in } \Omega;\ u = g \ \text{ on } \partial\Omega. \tag{1.4}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

This formulation discarded the approximation for $\boldsymbol{\nabla} u$ and approximated the flux $\boldsymbol{\sigma} := |\boldsymbol{\nabla} u|^{p-2}\boldsymbol{\nabla} u$ directly. Later, Creuse, Farhloul, and Paquet [14] numerically corroborated the order of convergence for lowest order Raviart–Thomas approximation proposed in [19] and used the hybridized version of the mixed method for the first time. Unfortunately, this method tends to break down when p is very close to 1. In [8], Burman and Ern proposed an interior penalty (IP) like DG method for the p-Laplacian problem by considering a discrete variational principle. Variational convergence was proven for the piecewise-linear case. Later, Buffa and Ortner [7] considered a more general minimization problem and proved the variational convergence of an IP-DG approximation, using piecewise polynomials of arbitrary but fixed degree, to the exact solution. Recently, Diening et al. [16] proposed a local discontinuous Galerkin (LDG) method based on the mixed formulation (1.1) with operators with a p-δ structure that is of the form $\mathcal{A}(\boldsymbol{\nabla} u) = (\delta + |\boldsymbol{\nabla} u|)^{p-2}\boldsymbol{\nabla} u$ and derived a nice a priori error analysis within the framework of Orlicz spaces for DG methods. Their analysis covers the p-Laplacian case when $\delta = 0$. However, in their numerical experiments, they take $\delta = 10^{-3}$ and consider a time-dependent case.

Even though many new methods were devised to solve p-Laplacian, to the authors' best knowledge, none of them considered high order approximations due to the low global regularity of the solution of p-Laplacian, except the work by Ainsworth and Kay [1]. Therein, they successfully developed the k-adaptive (k is polynomial degree) theory for the p-Laplacian and showed that the traditional view of avoiding the use of high order polynomial finite element methods is incorrect. In addition, the recent success of the h-adaptivity theory of HDG methods for linear steady-state diffusion by Cockburn and Zhang [13] recovers the optimal order of convergence $k + 1$ (measured in terms of degrees of freedom) even for the nonsmooth solutions. These results suggest that devising superconvergent HDG methods along with h/k-adaptivity theory is a promising approach. In this paper, we show the HDG methods can indeed achieve high order accuracy and superconvergence when the solution is sufficiently smooth. We also show that the method can capture "rough" solutions and deal with p in a very wide range even with uniform meshes. In addition, the postprocessing enables us to obtain a better approximation in both smooth and nonsmooth cases.

This paper is organized as follows. In section 2, we define our HDG methods, and in section 3, we describe two different variational approaches to solving the nonlinear equations. In section 4, we carry out numerical results to assess the performance of the methods. We end in section 5 with concluding remarks.

2. The HDG method.

In this section, we define the HDG method for the p-Laplacian and, following [24], we reformulate it as two equivalent minimization problems.

2.1. Notation.

We consider conforming triangulations of the domain Ω, $\mathcal{T}_h$. We denote the set of interior edges by $\mathcal{E}_h^o$ and the set of boundary edges by $\mathcal{E}_h^\partial$. The set $\mathcal{E}_h := \mathcal{E}_h^0 \cup \mathcal{E}_h^\partial$ is the collections of edges. Moreover, we denote $\boldsymbol{\nu}$ as the unit outer normal to the corresponding edge. For each element K in the triangulation $\mathcal{T}_h$, we denote by $\mathcal{P}^k(K)$ the space of polynomials of degree at most k defined on the element K. For each edge e in $\mathcal{E}_h$, $\mathcal{P}^k(e)$ is the space of polynomials of degree at most k defined on the edge e. Given an element K, $(\cdot,\cdot)_K$ and $\langle\cdot,\cdot\rangle_{\partial K}$ denote the $L^2(K)$ and $L^2(\partial K)$ product, respectively. Thus, for each ξ and ψ we define $(\xi,\psi)_{\mathcal{T}_h} := \sum_{K\in\mathcal{T}_h}(\xi,\psi)_K$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

and $\langle \xi, \psi\rangle_{\partial \mathcal{T}_h} := \sum_{K\in\mathcal{T}_h} \langle \xi, \psi\rangle_{\partial K}$. Finally, we define the following approximation spaces:

$$\boldsymbol{V}_h = \{\boldsymbol{v} \in [L^2(\mathcal{T}_h)]^2 : \quad \boldsymbol{v}|_K \in [\mathcal{P}^k(K)]^2 \quad \forall K \in \mathcal{T}_h\}, \tag{2.1a}$$

$$W_h = \{w \in L^2(\mathcal{T}_h) : \quad w|_K \in \mathcal{P}^k(K) \quad \forall K \in \mathcal{T}_h\}, \tag{2.1b}$$

$$M_h = \{\mu \in L^2(\mathcal{E}_h) : \quad \mu|_e \in \mathcal{P}^k(e) \quad \forall e \in \mathcal{E}_h\}, \tag{2.1c}$$

$$M_h(g) = \{\lambda \in M_h : \quad \langle \lambda, \mu\rangle_{\partial\Omega} = \langle g, \mu\rangle_{\partial\Omega} \quad \forall \mu \in M_h\}. \tag{2.1d}$$

2.2. Definition of the method. Next, we formulate the HDG scheme by discretizing (1.1). The method seeks an approximation $(\boldsymbol{\sigma}_h, \boldsymbol{q}_h, u_h, \widehat{u}_h)$ to the exact solution $(\boldsymbol{\sigma}, \boldsymbol{q}, u, u|_{\mathcal{E}_h})$ in the space $\boldsymbol{V}_h \times \boldsymbol{V}_h \times W_h \times M_h$ defined as the solution of

$$(\boldsymbol{q}_h, \boldsymbol{v})_{\mathcal{T}_h} - (u_h, \nabla\cdot \boldsymbol{v})_{\mathcal{T}_h} + \langle \widehat{u}_h, \boldsymbol{v}\cdot\boldsymbol{\nu}\rangle_{\partial\mathcal{T}_h} = 0, \tag{2.2a}$$

$$(\boldsymbol{\sigma}_h, \boldsymbol{v})_{\mathcal{T}_h} - (|\boldsymbol{q}_h|^{p-2}\boldsymbol{q}_h, \boldsymbol{v})_{\mathcal{T}_h} = 0, \tag{2.2b}$$

$$-(\boldsymbol{\sigma}_h, \boldsymbol{\nabla} w)_{\mathcal{T}_h} + \langle \widehat{\boldsymbol{\sigma}}_h\cdot\boldsymbol{\nu}, w\rangle_{\partial\mathcal{T}_h} = (f, w)_{\mathcal{T}_h}, \tag{2.2c}$$

$$\widehat{\boldsymbol{\sigma}}_h\cdot\boldsymbol{\nu} = \boldsymbol{\sigma}_h\cdot\boldsymbol{\nu} + \tau(u_h - \widehat{u}_h) \quad \text{on } \partial\mathcal{T}_h, \tag{2.2d}$$

and

$$\langle \widehat{\boldsymbol{\sigma}}_h\cdot\boldsymbol{\nu}, \mu\rangle_{\partial\mathcal{T}_h\setminus\partial\Omega} = 0, \tag{2.2e}$$

$$\langle \widehat{u}_h, \mu\rangle_{\partial\Omega} = \langle g, \mu\rangle_{\partial\Omega} \tag{2.2f}$$

$\forall(\boldsymbol{v}, w, \mu) \in \boldsymbol{V}_h \times W_h \times M_h$. We take the stabilization function τ to be positive on each of the edges in $\mathcal{E}_h$.

2.3. The minimization problems. Now, we express the HDG scheme as minimization problems; we follow [24].

2.3.1. The first minimization problem. Now, define the functional $\mathcal{J}_h : W_h \times M_h \to \mathbb{R}$ by

$$\mathcal{J}_h(u_h, \widehat{u}_h) = \frac{1}{p}(|\boldsymbol{q}_h|^p, 1)_{\mathcal{T}_h} + \frac{1}{2}\langle \tau(u_h - \widehat{u}_h), (u_h - \widehat{u}_h)\rangle_{\partial\mathcal{T}_h} - (f, u_h)_{\mathcal{T}_h},$$

where we consider $\boldsymbol{q}_h \in \boldsymbol{V}_h$ to be the function of u_h and $\widehat{u}_h$ given by the following *linear* equation on each element $K \subset \mathcal{T}_h$:

$$(\boldsymbol{q}_h, \boldsymbol{v})_K = (u_h, \nabla\cdot\boldsymbol{v})_K - \langle \widehat{u}_h, \boldsymbol{v}\cdot\boldsymbol{\nu}\rangle_{\partial K} \qquad \forall \boldsymbol{v} \in \boldsymbol{V}_h(K), \tag{2.3a}$$

where u_h and $\widehat{u}_h$ are independent to each other. In addition, the auxiliary function $\boldsymbol{\sigma}_h$ can be obtained from a decoupled local equation:

$$(\boldsymbol{\sigma}_h, \boldsymbol{v})_K - (|\boldsymbol{q}_h|^{p-2}\boldsymbol{q}_h, \boldsymbol{v})_K = 0 \qquad \forall \boldsymbol{v} \in \boldsymbol{V}_h(K). \tag{2.3b}$$

We have the following result.

THEOREM 2.1. *Let $(u_h, \widehat{u}_h)$ be a critical point of the functional $\mathcal{J}_h$ over the space $W_h \times M_h(g)$, and let $(\boldsymbol{\sigma}_h, \boldsymbol{q}_h)$ be defined by the linear equations* (2.3)*. Then $(\boldsymbol{\sigma}_h, \boldsymbol{q}_h, u_h, \widehat{u}_h)$ satisfies* (2.2) *defining the HDG approximation. Moreover, if the stabilization function τ is strictly positive on $\mathcal{E}_h$, the minimizer $(u_h, \widehat{u}_h)$ is unique.*

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Proof. If $(u_h, \widehat{u}_h)$ is a critical point of $\mathcal{J}_h$ that solves (2.3), we have $\forall\ (w,\mu) \in W_h \times M_h$ that

$$\begin{aligned}0 &= \delta\mathcal{J}_h((u_h,\widehat{u}_h);(w,\mu)) := \frac{d}{d\gamma}\mathcal{J}_h((u_h,\widehat{u}_h)+\gamma(w,\mu))|_{\gamma=0}\\ &= (|\boldsymbol{q}_h|^{p-2}\boldsymbol{q}_h, \delta\boldsymbol{q}_h((u_h,\widehat{u}_h);(w,\mu)))_{\mathcal{T}_h} + \langle\tau(u_h-\widehat{u}_h), w-\mu\rangle_{\partial\mathcal{T}_h} - (f,\omega)_{\mathcal{T}_h}\\ &= (\boldsymbol{\sigma}_h, \delta\boldsymbol{q}_h((u_h,\widehat{u}_h);(w,\mu)))_{\mathcal{T}_h} + \langle\tau(u_h-\widehat{u}_h), w-\mu\rangle_{\partial\mathcal{T}_h} - (f,\omega)_{\mathcal{T}_h}\end{aligned}$$

by taking $\boldsymbol{v} := \delta\boldsymbol{q}_h((u_h,\widehat{u}_h);(w,\mu))$ in (2.3b). If we now take the first variation on (2.3a) and set $\boldsymbol{v} := \boldsymbol{\sigma}_h$, we obtain that

$$\begin{aligned}0 = \delta\mathcal{J}_h((u_h,\widehat{u}_h);(w,\mu)) = &-\langle\widehat{\boldsymbol{\sigma}}_h\cdot\boldsymbol{\nu},\mu\rangle_{\partial\mathcal{T}_h\setminus\partial\Omega}\\ &+(\nabla\cdot\boldsymbol{\sigma}_h, w)_{\mathcal{T}_h} + \langle\tau(u_h-\widehat{u}_h), w\rangle_{\partial\mathcal{T}_h} - (f,w)_{\mathcal{T}_h}\end{aligned}$$

by the definition of $\widehat{\boldsymbol{\sigma}}_h$. Since w and μ are independent, the critical point $(u_h,\widehat{u}_h)$ as well as the function $(\boldsymbol{q}_h,\boldsymbol{\sigma}_h)$, given by (2.3), satisfy (2.2). On the other hand, any solution of (2.2) is a critical point of $\mathcal{J}_h$.

Next, we show that the functional $\mathcal{J}_h$ is strictly convex over $W_h \times M_h(g)$; as a consequence, the critical point $(u_h,\widehat{u}_h)$ would be *unique*. To prove this, let us mimic the argument in [9]. Assume (ω_1, m_1) and (ω_2, m_2) are two distinct elements in $W_h \times M_h(g)$ which determine $\boldsymbol{q}_1$, $\boldsymbol{q}_2$, respectively, via (2.3a). Then, given $\forall t \in]0,1[$,

$$\begin{aligned}\mathcal{J}_h(t(\omega_1,m_1)+(1-t)(\omega_2,m_2)) &= \frac{1}{p}(|t\boldsymbol{q}_1+(1-t)\boldsymbol{q}_2|^p,1)_{\mathcal{T}_h}\\ &+\frac{1}{2}\langle\tau(t\omega_1+(1-t)\omega_2-tm_1-(1-t)m_2)^2,1\rangle_{\partial\mathcal{T}_h} - (f, t\omega_1+(1-t)\omega_2)_{\mathcal{T}_h}.\end{aligned}$$

It is well known [9] that when $1<p<\infty$, the mapping $\xi\in\mathbb{R}^n : \xi \to \frac{1}{p}|\xi|^p$ is strictly convex. Hence, it implies that

$$\frac{1}{p}(|t\boldsymbol{q}_1+(1-t)\boldsymbol{q}_2|^p,1)_{\mathcal{T}_h} \le t\frac{1}{p}(|\boldsymbol{q}_1|^p,1)_{\mathcal{T}_h} + (1-t)\frac{1}{p}(|\boldsymbol{q}_2|^p,1)_{\mathcal{T}_h}.$$

Next since τ is strictly positive and we assume that the approximation space of scalar variable u is discontinuous across the interelement border, namely, $\omega_1|_{\mathscr{E}_h} \neq m_1$, $\omega_2|_{\mathscr{E}_h} \neq m_2$, we thus can utilize the strict convexity of quadratic function and deduce that

$$\mathcal{J}_h(t(\omega_1,m_1)+(1-t)(\omega_2,m_2)) < t\mathcal{J}_h(\omega_1,m_1)+(1-t)\mathcal{J}_h(\omega_2,m_2).$$

Finally, since W_h and $M_h(g)$ are convex sets, so is $W_h \times M_h(g)$. We can thus say that the functional $\mathcal{J}_h$ is strictly convex. In conclusion, its critical point is indeed the unique minimizer. Therefore, the solution of (2.2) is also unique. ∎

2.3.2. The second minimization problem. Let $J_h : M_h \to \mathbb{R}$ be given by

$$J_h(\widehat{u}_h) = \frac{1}{p}(|\boldsymbol{q}_h|^p,1)_{\mathcal{T}_h} + \frac{1}{2}\langle\tau(u_h-\widehat{u}_h),(u_h-\widehat{u}_h)\rangle_{\partial\mathcal{T}_h} - (f,u_h)_{\mathcal{T}_h},$$

where $(\boldsymbol{\sigma}_h,\boldsymbol{q}_h,u_h) \in \boldsymbol{V}_h \times \boldsymbol{V}_h \times W_h$ is the function of $\widehat{u}_h$ defined by the following equations on each element $K \subset \mathcal{T}_h$:

$$(\boldsymbol{q}_h,\boldsymbol{v})_K - (u_h, \nabla\cdot\boldsymbol{v})_K = -\langle\widehat{u}_h, \boldsymbol{v}\cdot\boldsymbol{\nu}\rangle_{\partial K}, \tag{2.4a}$$

$$(\boldsymbol{\sigma}_h,\boldsymbol{v})_K - (|\boldsymbol{q}_h|^{p-2}\boldsymbol{q}_h,\boldsymbol{v})_K = 0, \tag{2.4b}$$

$$(\nabla\cdot\boldsymbol{\sigma}_h, w)_K + \langle\tau u_h, w\rangle_{\partial K} = (f,w)_K + \langle\tau\widehat{u}_h, w\rangle_{\partial K}. \tag{2.4c}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

We have the following result.

THEOREM 2.2. *Let $\widehat{u}_h$ be a critical point of the functional J_h over the space $M_h(g)$, and let $(\boldsymbol{\sigma}_h, \boldsymbol{q}_h, u_h)$ be defined by the nonlinear equations* (2.4). *Then the function $(\boldsymbol{\sigma}_h, \boldsymbol{q}_h, u_h, \widehat{u}_h)$ satisfies* (2.2) *defining the HDG approximation. Moreover, the critical point $\widehat{u}_h$ is unique.*

Proof. Assume $\widehat{u}_h$ is a critical point of the functional J_h. Then $\forall \mu \in M_h(0)$, we have

$$
\begin{aligned}
0 &= \delta J_h(\widehat{u}_h;\mu) := \frac{d}{d\gamma} J_h(\widehat{u}_h + \gamma\mu)|_{\gamma=0} \\
&= (|\boldsymbol{q}_h|^{p-2}\boldsymbol{q}_h, \delta\boldsymbol{q}_h(\widehat{u}_h;\mu))_{\mathcal{T}_h} + \langle \tau(u_h - \widehat{u}_h), \delta u_h(\widehat{u}_h;\mu) - \mu\rangle_{\partial\mathcal{T}_h} - (f, \delta u_h(\widehat{u}_h;\mu))_{\mathcal{T}_h} \\
&= (\boldsymbol{\sigma}_h, \delta\boldsymbol{q}_h(\widehat{u}_h;\mu))_{\mathcal{T}_h} - (\nabla\cdot\boldsymbol{\sigma}_h, \delta u_h(\widehat{u}_h;\mu))_{\mathcal{T}_h} - \langle \tau(u_h - \widehat{u}_h), \mu\rangle_{\partial\mathcal{T}_h}
\end{aligned}
$$

by taking $\boldsymbol{v} := \delta\boldsymbol{q}_h(\widehat{u}_h;\mu)$ in (2.4b) and $\omega := \delta u_h(\widehat{u}_h;\mu)$ in (2.4c). Finally, we take the first variation on (2.4a) and then set $\boldsymbol{v} := \boldsymbol{\sigma}_h$ to obtain

$$
0 = \delta J_h(\widehat{u}_h;\mu) = -\langle \widehat{\boldsymbol{\sigma}}_h\cdot\boldsymbol{\nu}, \mu\rangle_{\partial\mathcal{T}_h\setminus\partial\Omega}
$$

by the definition of $\widehat{\boldsymbol{\sigma}}_h$, (2.2d). We conclude that $(\boldsymbol{\sigma}_h, \boldsymbol{q}_h, u_h, \widehat{u}_h)$ is a solution of (2.2). On the other hand, any solution of (2.2) is a critical point of J_h. In addition, by Theorem 2.1, (2.2) has a unique solution; therefore such critical point $\widehat{u}_h$ must be unique. This completes the proof. ∎

3. Solving the nonlinear equations. In this section, we describe gradient-descent type algorithms to solve our minimization problems.

3.1. The general form of the algorithm. We begin by showing the general flowchart for the gradient descent method we are using.

Algorithm 1. A general gradient descent algorithm.

Initialization: An initial guess $\boldsymbol{z}^{\mathbf{0}}$ and global tolerance $\delta > 0$;
for $i = 0, 1, \cdots$ **do**
  (1) Compute the gradient vector $\boldsymbol{g}^i = \boldsymbol{\nabla} f(\boldsymbol{z}^{\boldsymbol{i}})$.
  (2) Compute the (preconditioned) gradient $\boldsymbol{m}^i$ by solving $\mathcal{L}^{(i)}\boldsymbol{m}^i = \boldsymbol{g}^i$.
  (3) Compute the search direction $\boldsymbol{d}^i := -\boldsymbol{m}^i + \beta_i \boldsymbol{m}^{i-1}$, $\beta_0 = 0$.
  (4) Compute the stepsize α_i.
  (5) Set $\boldsymbol{z}^{i+1} := \boldsymbol{z}^{\boldsymbol{i}} + \alpha_i \boldsymbol{d}^i$.
  If $\|\boldsymbol{d}^i\| < \delta$, where $\|\cdot\|$ is a suitable norm, **return.**
end

In step (3), the gradient correction parameter β_i can have a significant impact on the performance of gradient descent type algorithm. Although in the existing literature (see, for example, [21]), there are many different options for choosing β_i, here we simply take $\beta_i = 0$ and leave other possibilities for future study. The optimal choice β_i seems to be crucial especially when the solution is not smooth.

In step (4), one could adopt the exact or inexact line search strategy (for example, see [23] for the CG method) to compute the step length. However, the line search requires a considerable number of evaluations of the functional, each of which requires the resolution of an HDG linear problem. Thus, it is very costly to conduct either exact or inexact line search in every global iteration. To overcome this difficulty, we take the approach of Shi and Shen [31] for estimating the step length; Zhou and Feng [33] adopted this idea for solving the p-Laplacian with the CG method. This

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

technique only carries out a line search on the very first iteration, $i = 0$ (we use the backtrack-Armijó strategy described below), and for $i \geq 1$, it defines it as follows:

$$\alpha_i = \frac{-(\boldsymbol{g}^i)^T \boldsymbol{d}^i}{L_i |\boldsymbol{d}^i|^2}, \qquad L_i = \frac{|\boldsymbol{g}^i - \boldsymbol{g}^{i-1}|}{|\boldsymbol{z}^i - \boldsymbol{z}^{i-1}|}. \tag{3.1}$$

Here, $|\cdot|$ denotes the Euclidean norm for vectors.

Algorithm 2. Backtrack-Armijó line search.

Initialization: An initial stepsize α^0, a starting point $\boldsymbol{z}^0$, search direction $\boldsymbol{d}$, parameters $c_1, c_2 \in (0,1)$, and line search tolerance δ_l;
while $\alpha^i > \delta_l$ **do**
  $\boldsymbol{z}^n = \boldsymbol{z}^0 + \alpha^i \boldsymbol{d}$;
  if $J_h(\boldsymbol{z}^n) < J_h(\boldsymbol{z}^0) + c_2 \alpha^i [\boldsymbol{\nabla} J_h(\boldsymbol{z}^0)]^T \boldsymbol{d}$ **then**
    Return;
  else
    $\alpha^i = c_1 \alpha^i$;
  end
end

3.2. The four algorithms. Next, we specify the choices of $\boldsymbol{z}, f, \mathcal{L}^{(i)}$, and $\|\cdot\|$ that define our four algorithms, namely, Algorithm I, Algorithm II, and their preconditioned versions (Table 1).

TABLE 1
Description of the four minimization algorithms.

Algorithm	$\boldsymbol{z}$	f	$\mathcal{L}^i$	$\|\cdot\|$
Algorithm I	$[\widehat{u}_h]$	J_h	Id	$\|\cdot\|_{\ell^2}$
Preconditioned Algorithm I	$[\widehat{u}_h]$	J_h	$\mathrm{B}_{\mathrm{I}}^{(i)}$	$\|\cdot\|_{\mathrm{I}}$
Algorithm II	$([u_h], [\widehat{u}_h])$	$\mathcal{J}_h$	Id	$\|\cdot\|_{\ell^2}$
Preconditioned Algorithm II	$([u_h], [\widehat{u}_h])$	$\mathcal{J}_h$	$\mathrm{B}_{\mathrm{II}}^{(i)}$	$\|\!\|\!\|\cdot\|\!\|\!\|_{\mathrm{II}}$

3.2.1. Algorithm I. Note that by Theorem 2.2, the gradient vector $\boldsymbol{g}^i$ satisfies

$$[\mu]^t \boldsymbol{g}^i = -\langle [\![\widehat{\boldsymbol{\sigma}}_h^i]\!], \mu \rangle_{\mathscr{E}_h^o}.$$

Note that although here we only work with degrees of freedom $\widehat{u}_h$, we still have to solve the local nonlinear equations (2.4b), embedded in steps (1) and (4). Therefore, this algorithm may still suffer the instability caused by the linearization required by standard fixed point iterations.

3.2.2. Algorithm II. Note that by Theorem 2.1, the gradient vector $\boldsymbol{g}^i$ has two components, namely,

$$\begin{aligned} [0, \mu]^t \boldsymbol{g}^i &= -\langle [\![\widehat{\boldsymbol{\sigma}}_h^i]\!], \mu \rangle_{\mathscr{E}_h^o}, \\ [\omega, 0]^t \boldsymbol{g}^i &= -(\nabla \cdot \boldsymbol{\sigma}_h^i, \omega)_{\mathcal{T}_h} - \langle \tau(u_h^i - \widehat{u}_h^i), w \rangle_{\partial \mathcal{T}_h} + (f, w)_{\mathcal{T}_h}. \end{aligned}$$

Note also that one of the distinctive features of Algorithm II is that the local solver (2.3a) is linear. Compared to Algorithm I, we no longer encounter the potential unstability of the solvers for the local problems. However, it does have significantly more degrees of freedom.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

3.2.3. Preconditioning. We are going to use the so-called frozen coefficient preconditioner described in [17], where the authors claim that this type of preconditioner usually yields a mesh-independent global convergence rate. In fact, the weighted-norm preconditioner for the p-Laplacian equation developed in [23] can be categorized as a special frozen coefficient preconditioner and their numerical results do display a mesh-independent convergence rate for CG piecewise-linear approximations.

Next. we describe the frozen coefficient preconditioners $B_{\rm I}^{(i)}$ and $B_{\rm II}^{(i)}$ for our algorithms when $p > 2$; the modification for the case $1 < p < 2$ is described in [23].

DEFINITION 3.1 (preconditioning for HDG methods). *Given $\boldsymbol{q}_h^i \in \boldsymbol{V}_h$, the gradient vector $\boldsymbol{g}^i = [\boldsymbol{g}_u^i, \boldsymbol{g}_{\widehat{u}}^i]$, $p > 2$, and $\epsilon > 0$, we define $(\boldsymbol{\mathcal{S}}^i, \boldsymbol{\mathcal{Q}}^i, \mathcal{U}^i, \widehat{\mathcal{U}}^i) \in \boldsymbol{V}_h \times \boldsymbol{V}_h \times W_h \times M_h(0)$ as the solution of*

$$(\boldsymbol{\mathcal{Q}}^i, \boldsymbol{v})_{\mathcal{T}_h} - (\mathcal{U}^i, \nabla \cdot \boldsymbol{v})_{\mathcal{T}_h} = -\langle \widehat{\mathcal{U}}^i, \boldsymbol{v} \cdot \boldsymbol{\nu} \rangle_{\partial \mathcal{T}_h}, \tag{3.2a}$$

$$(\boldsymbol{\mathcal{S}}^i, \boldsymbol{v})_{\mathcal{T}_h} - ((\epsilon + |\boldsymbol{q}_h^i|^{p-2})\boldsymbol{\mathcal{Q}}^i, \boldsymbol{v})_{\mathcal{T}_h} = 0, \tag{3.2b}$$

$$(\nabla \cdot \boldsymbol{\mathcal{S}}^i, w)_{\mathcal{T}_h} + \langle \tau(\mathcal{U}^i - \widehat{\mathcal{U}}^i), w \rangle_{\partial \mathcal{T}_h} = [\omega]^t \boldsymbol{g}_u^i, \tag{3.2c}$$

$$\widehat{\boldsymbol{\mathcal{S}}}^i \cdot \boldsymbol{\nu} = \boldsymbol{\mathcal{S}}^i \cdot \boldsymbol{\nu} + \tau(\mathcal{U}^i - \widehat{\mathcal{U}}^i), \tag{3.2d}$$

$$\langle \widehat{\boldsymbol{\mathcal{S}}}^i \cdot \boldsymbol{\nu}, \mu \rangle_{\partial \mathcal{T}_h \setminus \partial \Omega} = [\mu]^t \boldsymbol{g}_{\widehat{u}}^i, \tag{3.2e}$$

$$\langle \widehat{\mathcal{U}}^i, \mu \rangle_{\partial \Omega} = 0 \tag{3.2f}$$

$\forall$ test functions $(\boldsymbol{v}, w, \mu) \in \boldsymbol{V}_h \times W_h \times M_h$. The above equations can be viewed in operator form $\mathcal{L}^{(i)} \boldsymbol{m}^i = \boldsymbol{g}^i$. For the cases $\mathcal{L}^{(i)} = B_{\rm I}^{(i)}$ and $\mathcal{L}^{(i)} = B_{\rm II}^{(i)}$, we specify $[\omega]^t \boldsymbol{g}_u^i$, $[\mu]^t \boldsymbol{g}_{\widehat{u}}^i$, and $\boldsymbol{m}^i$ explicitly in Table 2.

TABLE 2
Description of the two preconditioning operators.

$\mathcal{L}^{(i)}$	$[\omega]^t \boldsymbol{g}_u^i$	$[\mu]^t \boldsymbol{g}_{\widehat{u}}^i$	$\boldsymbol{m}^i$
$B_{\rm I}^{(i)}$	0	$-\langle \widehat{\boldsymbol{\sigma}}_h^i \cdot \boldsymbol{\nu}, \mu \rangle_{\partial \mathcal{T}_h \setminus \partial \Omega}$	$[\widehat{\mathcal{U}}^i]$
$B_{\rm II}^{(i)}$	$-(\nabla \cdot \boldsymbol{\sigma}_h^i, w)_{\mathcal{T}_h}$ $-\langle \tau(u_h^i - \widehat{u}_h^i), w \rangle_{\partial \mathcal{T}_h} + (f, w)_{\mathcal{T}_h}$	$-\langle \widehat{\boldsymbol{\sigma}}_h^i \cdot \boldsymbol{\nu}, \mu \rangle_{\partial \mathcal{T}_h \setminus \partial \Omega}$	$([\mathcal{U}^i], [\widehat{\mathcal{U}}^i])$

Note that since the function $\boldsymbol{q}_h^i$ is data in (3.2) defining the preconditioning, they can be viewed as defining the HDG approximation for a linear, steady-state diffusion problem [10]. In addition, the presence of ϵ ensures the positivity of the coefficient $(\epsilon + |\boldsymbol{q}_h^i|^{p-2})$, when $\boldsymbol{q}_h^i = \boldsymbol{0}$. Consequently, the well-posedness of the preconditioning problem follows immediately.

COROLLARY 3.2. *For any strictly positive ϵ and any strictly positive stabilization function τ, the approximate solution given by* (3.2) *is well defined. In particular, $\boldsymbol{m}^i = \boldsymbol{0}$ if and only if $\boldsymbol{g}^i = \boldsymbol{0}$.*

Remark 3.3. We add the small positive constant ϵ only in the preconditioning equations (3.2), not in the numerical scheme. Therefore, the presence of ϵ does not change the solution of the HDG methods defined in (2.2).

As a consequence of Theorems 2.1 and 2.2, when $\boldsymbol{m}^i = \boldsymbol{0}$, for preconditioned Algorithm I, J_h attains a critical point at $\widehat{u}_h^i$, and for preconditioned Algorithm II, $\mathcal{J}_h$ attains its unique minimizer $(u_h^i, \widehat{u}_h^i)$. Therefore, the use of the preconditioners does not alter the convergence to the minimizers of our functional.

It remains to define the norms $\|\cdot\|_{\rm I}$ and $|||\cdot|||_{\rm II}$. We define them as the energy norm of the problem defining the preconditionners.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

DEFINITION 3.4. *Given* $\boldsymbol{q}_h^i \in \boldsymbol{V}_h$, $\boldsymbol{g}^i$, $p > 2, \epsilon > 0$, *let* $(\boldsymbol{\mathcal{S}}^i, \boldsymbol{\mathcal{Q}}^i, \mathcal{U}^i, \widehat{\mathcal{U}}^i) \in \boldsymbol{V}_h \times \boldsymbol{V}_h \times W_h \times M_h(0)$ *be the solution of* (3.2) *in the* i*th iteration. Then, we set*

$$\|[\widehat{\mathcal{U}}^i]\|_{\mathrm{I}}^2 := ((\epsilon + |\boldsymbol{q}_h^i|^{p-2})\boldsymbol{\mathcal{Q}}^i, \boldsymbol{\mathcal{Q}}^i)_{\mathcal{T}_h} + \langle \tau(\mathcal{U}^i - \widehat{\mathcal{U}}^i), (\mathcal{U}^i - \widehat{\mathcal{U}}^i)\rangle_{\partial\mathcal{T}_h},$$

$$\left|\!\left|\!\left|([\mathcal{U}^i], [\widehat{\mathcal{U}}^i])\right|\!\right|\!\right|_{\mathrm{II}}^2 := ((\epsilon + |\boldsymbol{q}_h^i|^{p-2})\boldsymbol{\mathcal{Q}}^i, \boldsymbol{\mathcal{Q}}^i)_{\mathcal{T}_h} + \langle \tau(\mathcal{U}^i - \widehat{\mathcal{U}}^i), (\mathcal{U}^i - \widehat{\mathcal{U}}^i)\rangle_{\partial\mathcal{T}_h}.$$

3.3. Postprocessing. Next, let us describe a local postprocessing technique used in [11, 12]. The resulting solution u_h^* provides a better approximation for the scalar variable u. In particular, the function u_h^* is defined as follows. On each element $K \subset \mathcal{T}_h$,

$$(\boldsymbol{\nabla} u_h^*, \boldsymbol{\nabla} w)_K = -(\boldsymbol{q}_h, \boldsymbol{\nabla} w)_K \quad \forall w \in \mathcal{P}^{k+1}(K), \tag{3.3a}$$

$$(u_h^*, 1)_K = (u_h, 1)_K. \tag{3.3b}$$

In [2], the authors revealed the extra information contained in hybridization and used it to enhance the accuracy of the scalar unknown. This was later achieved for the HDG methods in [11]. In our nonlinear setting, we discover a new advantage, namely, that it can be used to provide good initial guesses. In most of the existing literature, the initial guesses of nonlinear iterative solvers are taken to be zero, or are generated randomly or by using the approximate solution of a "similar" linear equation. When using a piecewise-constant approximation, these strategies usually work well. However, for high order accurate methods, a good initial guess is important to ensure the convergence of iterative solvers, especially when dealing with nonsmooth solutions. Therefore, it is reasonable to choose an initial guess which approximates the exact solution as well as possible. We display the flowchart of our algorithm as follows.

Algorithm 3. Generating initial guess $(u_h^0, \widehat{u}_h^0)$ using when $k \geq 1$.

Initialization: Given the mesh $\mathcal{T}_h$, solve the linear ($p := 2$) HDG system (2.2) using piecewise-constant polynomials. The solution $(u_h, \widehat{u}_h)$ is set as the initial guess $(u_h^0, \widehat{u}_h^0)$;

for $i = 0, 1, \ldots, k$ **do**

(1) Solve the HDG scheme for the p-Laplacian with polynomials of degree i for all variables starting from the initial guess $(u_h^0, \widehat{u}_h^0)$;

(2) Compute the postprocessing $u_h^* \in \mathcal{P}^{i+1}(\mathcal{T}_h)$ by using (3.3).

(3) For each edge $e \in \mathcal{E}_h$, set $\widehat{u}_h^*|_e \in \mathcal{P}^{i+1}(e)$ to be the average of u_h^* on both sides of the edge e.

(4) Update the initial guess $(u_h^0, \widehat{u}_h^0) \leftarrow (u_h^*, \widehat{u}_h^*)$

end

4. Numerical results. In this section, we conduct several numerical experiments to explore the accuracy and efficiency of our methods. In subsection 4.2, we explore the accuracy of the HDG method when the solution is sufficiently regular. In subsection 4.3, we explore the performance of the preconditioners. Finally in subsection 4.4, we explore the performance of the HDG method on more difficult problems where solutions display near-zero or strong gradients.

4.1. Implementation considerations. Let us describe some implementation details in order to make our presentation transparent.

Initial guess. Our experience indicates that the use of Algorithm 3 for computing the initial guess does increase the chance of convergence when the solution is "rough." For smooth solutions, it is not necessary to apply it.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Line search. Next, we briefly discuss how to choose the parameters of Algorithm 2, the backtrack-Armijó line search. The initial stepsize α^0 must be properly chosen to avoid a low convergence rate or redundant backtracking. The parameter c_1 is the stepsize contraction parameter that specifies how much we reduce the stepsize in each line search iteration. Increasing c_1 improves the line search accuracy but also increases the number of line search iterations. Finally, a suitable choice of c_2 guarantees a sufficient decrease of the functional. In our experience, it is reasonable to take $c_1 = 0.6$, $c_2 = 10^{-4}$, $\alpha^0 = 1$ and the line search tolerance $\delta_l = 10^{-4}$ when the solution is smooth. When the solution is of low regularity, in order to increase the chance of convergence, we have to choose a larger c_1 to improve the accuracy of line search and a larger ϵ to compensate for the possible degeneracy of the flux in the preconditioners.

Iterative solvers and CPU time. For Algorithm I and preconditioned Algorithm I, we solve the local nonlinear equations by the *Newton–Raphson* method with tolerance 10^{-10}; the initial guess is obtained from the previous global iteration. Furthermore, when solving the linear system associated with the preconditionner, we adopt the MATLAB backslash built-in function, which employs the linear time multifrontal method. Our code is written in MATLAB and we take the measurement of CPU time on the node labq63 in the Minnesota Supercomputing Institute, which has an Intel Xeon 2.66-GHz CPU core and 16 GB memory. We adopt the average CPU time of three runs.

4.2. Accuracy of the HDG method. In this section, we carry out several numerical experiments to explore the convergence properties of our HDG scheme when (i) the solution is sufficiently regular and (ii) its gradient does not vanish in the entire domain. We take the domain $\Omega = (0,1) \times (0,1)$ and set the stabilization function τ identically equal to 1. In each of the experiments, we monitor the behavior of the following errors:

$$e_u = P_h u - u_h, \;\; e_{\boldsymbol{q}} = \Pi_h \boldsymbol{q} - \boldsymbol{q}_h, \;\; e_{\boldsymbol{\sigma}} = \Pi_h \boldsymbol{\sigma} - \boldsymbol{\sigma}_h, \;\; e_{u^*} = L_h u - u_h^*.$$

Here, P_h, Π_h, and L_h are L^2 projections onto $\mathcal{P}^k(\mathcal{T}_h)$, $[\mathcal{P}^k(\mathcal{T}_h)]^2$, and $\mathcal{P}^{k+1}(\mathcal{T}_h)$, respectively. We set the global tolerance δ to be 10^{-12}. Since the gradient of solution does not vanish inside the domain, we simply take $\epsilon = 0$ for the preconditioner.

4.2.1. Smooth case. In our first experiment, we test the performance of our HDG scheme for the case when u is smooth. To do this, we choose f and g so that the exact solution is

$$u(x,y) = ((x+1)^2 + (y+1)^2)^{\frac{p-2}{2(p-1)}}. \tag{4.1}$$

In Tables 3 and 4 we present the numerical results for the cases $p = 1.5$ and $p = 3$. In both tables, we observe $k+1$ order of convergence for $\boldsymbol{q}_h, \boldsymbol{\sigma}_h$, and u_h, when $k \geq 0$ in all norms. Moreover, the postprocessed solution u_h^* exhibits order $k+2$ when $k \geq 1$. In conclusion, we confirm that the behavior of our HDG scheme is the same as that of the HDG method for the linear steady-state diffusion problem in [10] for smooth solutions.

4.2.2. Less regular case. In our second experiment, we test the performance of the method for the case when u has limited regularity. To achieve this, we choose f and g to satisfy the exact solution:

$$u(x,y) = \frac{p-1}{p+s}\left(\frac{1}{2+s}\right)^{\frac{1}{p-1}} (1 - (x^2+y^2))^{\frac{p+s}{2(p-1)}}). \tag{4.2}$$

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

TABLE 3
History of convergence for $h = 0.707, \Omega = (0,1)^2$ and smooth case with $p = 1.5$.

		$\|e_{\boldsymbol{\sigma}}\|_{L^p(\Omega)}$		$\|e_{\boldsymbol{q}}\|_{L^p(\Omega)}$		$\|e_u\|_{L^p(\Omega)}$		$\|e_{u^*}\|_{L^p(\Omega)}$	
k	Mesh	Error	Order	Error	Order	Error	Order	Error	Order
0	h	2.87E-02	-	2.67E-02	-	3.56E-03	-	4.43E-03	-
	$h/2$	1.39E-02	0.95	1.35E-02	0.99	1.70E-03	1.07	1.82E-03	1.29
	$h/4$	7.55E-03	0.98	6.77E-03	1.00	8.40E-04	1.01	8.55E-04	1.09
	$h/8$	3.79E-03	0.99	3.38E-03	1.00	4.19E-04	1.00	4.21E-04	1.02
1	h	3.10E-03	-	2.28E-03	-	2.37E-04	-	2.11E-04	-
	$h/2$	8.49E-04	1.88	5.88E-04	1.95	5.09E-05	2.22	2.88E-05	2.87
	$h/4$	2.21E-04	1.94	1.49E-04	1.98	1.14E-06	2.16	3.84E-06	2.91
	$h/8$	5.62E-05	1.97	3.76E-05	1.99	2.72E-06	2.07	4.97E-07	2.95
2	h	2.81E-04	-	2.30E-04	-	3.54E-05	-	1.12E-06	-
	$h/2$	3.89E-05	2.86	3.15E-05	2.87	4.30E-06	3.04	7.47E-07	3.91
	$h/4$	5.06E-06	2.94	4.08E-06	2.95	5.25E-07	3.03	4.76E-08	3.97
	$h/8$	6.44E-07	2.97	5.17E-07	2.98	6.46E-08	3.02	2.99E-09	3.99
3	h	2.33E-05	-	1.94E-05	-	1.56E-06	-	9.66E-07	-
	$h/2$	1.63E-06	3.84	1.31E-06	3.89	9.46E-08	4.04	3.44E-08	4.81
	$h/4$	1.06E-08	3.94	8.38E-08	3.96	5.98E-09	3.98	1.11E-09	4.95
	$h/8$	6.72E-09	3.98	5.29E-09	3.99	3.79E-10	3.98	3.47E-11	5.00
		$\|e_{\boldsymbol{\sigma}}\|_{L^1(\Omega)}$		$\|e_{\boldsymbol{q}}\|_{L^1(\Omega)}$		$\|e_u\|_{L^1(\Omega)}$		$\|e_{u^*}\|_{L^1(\Omega)}$	
k	Mesh	Error	Order	Error	Order	Error	Order	Error	Order
0	h	2.83E-02	-	2.57E-02	-	3.46E-03	-	3.53E-03	-
	$h/2$	1.46E-02	0.95	1.29E-02	1.00	1.66E-03	1.06	1.66E-03	1.09
	$h/4$	7.40E-03	0.98	6.43E-03	1.00	8.22E-04	1.01	8.22E-04	1.01
	$h/8$	3.72E-03	0.99	3.21E-03	1.00	4.10E-04	1.00	4.10E-04	1.00
1	h	2.67E-03	-	1.93E-03	-	1.99E-04	-	1.59E-04	-
	$h/2$	7.30E-04	1.87	5.06E-04	1.93	4.43E-05	2.17	2.21E-05	2.85
	$h/4$	1.89E-04	1.95	1.29E-04	1.97	9.55E-06	2.22	3.02E-06	2.87
	$h/8$	4.82E-05	1.98	3.28E-05	1.99	2.24E-06	2.09	3.99E-07	2.92
2	h	2.55E-04	-	1.97E-04	-	3.39E-05	-	8.44E-06	-
	$h/2$	3.48E-04	2.87	2.71E-05	2.86	3.96E-06	3.10	5.40E-07	3.96
	$h/4$	4.50E-05	2.95	3.52E-06	2.94	4.68E-07	3.08	3.44E-08	3.97
	$h/8$	5.71E-06	2.98	4.48E-07	2.97	5.62E-08	3.06	2.16E-09	3.99
3	h	1.97e-05	-	1.67E-05	-	1.11E-06	-	8.72E-07	-
	$h/2$	1.38E-06	3.84	1.07E-06	3.96	6.40E-08	4.12	3.14E-08	4.79
	$h/4$	8.90E-08	3.95	6.78E-08	3.99	4.31E-09	3.89	1.01E-09	4.96
	$h/8$	5.60E-09	3.99	4.25E-09	3.99	2.85E-10	3.92	3.14E-11	5.00

We take this example from [3] by setting $s = 0$ and $p = 3$. It can be verified that u lies in $W^{3,1}(\Omega)$ but not in $W^{3,p}(\Omega)$ or $H^3(\Omega)$. Hence, we expect optimal $\mathcal{O}(h^3)$ convergence for u_h and $\mathcal{O}(h^2)$ convergence for $\boldsymbol{q}_h$ only in the L^1-norm but not in the L^p-norm or in the L^2-norm. The numerical results in Table 5 confirm this behavior. In particular, when using polynomials of degree one for $\boldsymbol{V}_h, W_h$, and M_h, we observe that the approximation of gradient $\boldsymbol{q}_h$ exhibits optimal $\mathcal{O}(h^2)$ convergence in the L^1-norm and that the postprocessing u_h^* gains an extra order of convergence only in the L^1-norm. We further observe that when using polynomials of degree two, u_h exhibits optimal $\mathcal{O}(h^3)$ convergence only in the L^1-norm. However, the postprocessing u_h^* still converges with a rate $\mathcal{O}(h^3)$, as expected.

4.3. The performance of solvers. We begin exploring the effect of the preconditioners $B_{\mathrm{I}}^{(i)}$ and $B_{\mathrm{II}}^{(i)}$. Then, we compare the performance of the preconditioned Algorithms I and II in terms of convergence speed, size of global degrees of freedom and robustness with respect to variations in the values of p. We set the stabilization function τ to be identically equal to 1.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

TABLE 4
History of convergence for $h = 0.707, \Omega = (0,1)^2$ and smooth case with $p = 3$.

		$\|e_{\boldsymbol{\sigma}}\|_{L^p(\Omega)}$		$\|e_{\boldsymbol{q}}\|_{L^p(\Omega)}$		$\|e_u\|_{L^p(\Omega)}$		$\|e_{u^*}\|_{L^p(\Omega)}$	
k	Mesh	Error	Order	Error	Order	Error	Order	Error	Order
0	h	8.42E-03	-	1.35E-02	-	5.62E-03	-	6.33E-03	-
	$h/2$	4.77E-03	0.82	8.37E-03	0.69	2.90E-03	0.94	3.01E-03	1.07
	$h/4$	2.58E-03	0.89	4.79E-03	0.81	1.48E-03	0.97	1.49E-03	1.01
	$h/8$	1.35E-03	0.94	2.56E-03	0.90	7.46E-04	0.99	7.48E-04	1.00
1	h	2.04E-03	-	4.06E-03	-	5.36E-04	-	5.45E-04	-
	$h/2$	5.74E-04	1.83	1.18E-03	1.79	1.02E-04	2.38	7.90E-05	2.79
	$h/4$	1.52E-04	1.91	3.18E-04	1.89	2.15E-05	2.25	1.07E-05	2.89
	$h/8$	3.93E-05	1.96	8.27E-05	1.94	4.92E-06	2.13	1.39E-06	2.94
2	h	9.45E-05	-	1.91E-04	-	2.72E-05	-	1.19E-05	-
	$h/2$	1.42E-05	2.74	2.96E-05	2.69	3.35E-06	3.02	8.03E-07	3.89
	$h/4$	1.94E-06	2.87	4.08E-06	2.86	4.12E-07	3.02	5.17E-08	3.96
	$h/8$	2.53E-07	2.94	5.35E-07	2.93	5.11E-08	3.01	3.27E-09	3.98
3	h	8.64E-06	-	1.89E-05	-	1.11E-06	-	1.04E-06	-
	$h/2$	6.68E-07	3.69	1.42E-06	3.74	6.32E-08	4.14	3.27E-08	5.00
	$h/4$	4.57E-08	3.87	9.63E-08	3.89	3.78E-09	4.06	9.76E-10	5.07
	$h/8$	2.98E-09	3.94	6.24E-09	3.95	2.29E-10	4.04	2.96E-11	5.05
		$\|e_{\boldsymbol{\sigma}}\|_{L^1(\Omega)}$		$\|e_{\boldsymbol{q}}\|_{L^1(\Omega)}$		$\|e_u\|_{L^1(\Omega)}$		$\|e_{u^*}\|_{L^1(\Omega)}$	
k	Mesh	Error	Order	Error	Order	Error	Order	Error	Order
0	h	7.31E-03	-	1.17E-02	-	5.00E-03	-	5.00E-03	-
	$h/2$	3.90E-03	0.91	6.90E-03	0.76	2.45E-03	1.03	2.45E-03	1.03
	$h/4$	2.03E-03	0.94	3.81E-03	0.86	1.22E-03	1.01	1.22E-03	1.01
	$h/8$	1.05E-03	0.96	2.01E-03	0.92	6.10E-04	1.00	6.10E-03	1.00
1	h	1.74E-03	-	3.10E-03	-	4.07E-04	-	3.25E-04	-
	$h/2$	4.71E-04	1.88	8.49E-04	1.87	7.28E-05	2.48	4.69E-05	2.79
	$h/4$	1.22E-04	1.95	2.22E-04	1.93	1.34E-05	2.44	6.37E-06	2.88
	$h/8$	3.12E-05	1.97	5.70E-05	1.97	2.95E-06	2.19	8.31E-07	2.94
2	h	8.08E-05	-	1.44E-04	-	2.71E-05	-	8.66E-06	-
	$h/2$	1.11E-05	2.87	2.07E-05	2.79	2.90E-06	3.22	6.07E-07	3.83
	$h/4$	1.39E-06	2.99	2.73E-06	2.93	3.43E-07	3.08	3.79E-08	4.00
	$h/8$	1.74E-07	3.00	3.48E-07	2.97	4.15E-08	3.05	2.30E-09	4.04
3	h	6.00E-06	-	1.23E-05	-	6.28E-07	-	4.30E-07	-
	$h/2$	4.39E-07	3.77	9.43E-07	3.71	2.60E-08	4.59	1.61E-08	4.74
	$h/4$	2.97E-08	3.89	6.43E-08	3.87	1.47E-09	4.14	5.84E-10	4.79
	$h/8$	1.93E-09	3.94	4.20E-09	3.94	9.83E-11	3.91	1.94E-11	4.91

The notation we use in Tables 6 and 7 is as follows. We denote by N_{DOF} the number of degrees of freedom or independent arguments of the functional, by N_{Iter} we denote the number of iterations for convergence, and by CPU we denote the total CPU time measured in seconds. For the two preconditioned algorithms, we define three additional measurements: CPU_{prec} refers to the CPU time in seconds spent for the preconditioning, CPU_{line} refers to the CPU time in seconds for performing the line search procedure, and CPU_{local} refers to the percentage of CPU time for the computation of the local solvers with respect to the total CPU time.

4.3.1. Effect of preconditioner. Let us first consider Algorithm I with and without preconditioning. In Table 6, we see that the preconditioned Algorithm I has many fewer iterations and significantly less total CPU time. Furthermore, the number of iterations required for the preconditioned Algorithm I exhibits a solid h-independent and a surprising k-independent behavior. In contrast, Algorithm I needs about three times more iterations to converge when the meshsize is halved. On the other hand, the CPU time indicates that the computational complexity of Algorithm I is of order N^2 (where N is the size of globally coupled degrees of freedom), whereas that of the preconditioned Algorithm I is only of order N.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

TABLE 5
History of convergence for $h = 0.707, \Omega = (0,1)^2$, and less regular case with $p = 3, s = 0$.

		$\|e_{\boldsymbol{\sigma}}\|_{L^p(\Omega)}$		$\|e_{\boldsymbol{q}}\|_{L^p(\Omega)}$		$\|e_u\|_{L^p(\Omega)}$		$\|e_{u^*}\|_{L^p(\Omega)}$	
k	Mesh	Error	Order	Error	Order	Error	Order	Error	Order
0	h	4.35E-02	-	4.10E-02	-	6.49E-02	-	6.55E-02	-
	$h/2$	2.51E-02	0.79	2.94E-02	0.48	3.21E-02	1.01	3.21E-02	1.03
	$h/4$	1.34E-02	0.90	1.78E-02	0.72	1.60E-02	1.00	1.60E-02	1.01
	$h/8$	6.95E-03	0.95	9.94E-03	0.84	7.99E-03	1.00	7.99E-03	1.00
	$h/16$	3.54E-03	0.97	5.28E-03	0.91	3.99E-03	1.00	3.99E-03	1.00
1	h	1.26E-02	-	2.10E-02	-	2.93E-03	-	2.75E-03	-
	$h/2$	3.42E-03	1.88	7.76E-03	1.44	6.49E-04	2.18	5.40E-04	2.35
	$h/4$	9.09E-04	1.91	2.82E-03	1.46	1.41E-04	2.20	1.01E-04	2.42
	$h/8$	2.37E-04	1.94	1.04E-03	1.44	3.16E-05	2.15	1.87E-05	2.42
	$h/16$	6.13E-05	1.95	3.94E-04	1.40	7.43E-06	2.10	3.57E-06	2.39
2	h	1.08E-03	-	3.61E-03	-	3.25E-04	-	4.90E-04	-
	$h/2$	2.90E-04	1.91	1.41E-03	1.36	6.95E-05	2.23	1.06E-04	2.20
	$h/4$	7.99E-05	1.86	5.63E-04	1.33	1.50E-05	2.21	2.31E-05	2.21
	$h/8$	2.28E-05	1.81	2.30E-04	1.29	3.25E-06	2.20	5.09E-06	2.19
	$h/16$	6.72E-06	1.76	9.54E-05	1.27	7.13E-07	2.19	1.12E-06	2.18
		$\|e_{\boldsymbol{\sigma}}\|_{L^2(\Omega)}$		$\|e_{\boldsymbol{q}}\|_{L^2(\Omega)}$		$\|e_u\|_{L^2(\Omega)}$		$\|e_{u^*}\|_{L^2(\Omega)}$	
k	Mesh	Error	Order	Error	Order	Error	Order	Error	Order
0	h	4.16E-02	-	4.06E-02	-	6.42E-02	-	6.45E-02	-
	$h/2$	2.40E-02	0.79	2.74E-02	0.57	3.17E-02	1.02	3.17E-02	1.02
	$h/4$	1.27E-02	0.91	1.58E-02	0.80	1.58E-02	1.00	1.58E-02	1.01
	$h/8$	6.57E-03	0.96	8.51E-03	0.89	7.87E-03	1.00	7.87E-03	1.00
	$h/16$	3.33E-03	0.98	4.43E-03	0.94	3.93E-03	1.00	3.93E-03	1.00
1	h	1.14E-02	-	1.70E-02	-	2.53E-03	-	2.27E-03	-
	$h/2$	3.01E-03	1.92	5.43E-03	1.64	5.28E-04	2.26	4.07E-04	2.48
	$h/4$	7.76E-04	1.96	1.64E-04	1.73	1.10E-04	2.26	6.69E-05	2.61
	$h/8$	1.97E-04	1.98	4.82E-05	1.76	2.44E-05	2.18	1.04E-05	2.67
	$h/16$	4.97E-05	1.99	1.42E-05	1.77	5.73E-06	2.09	1.61E-06	2.71
2	h	8.49E-04	-	2.09E-03	-	2.70E-04	-	2.96E-04	-
	$h/2$	1.87E-04	2.18	6.45E-04	1.70	4.67E-05	2.53	5.10E-05	2.53
	$h/4$	4.23E-05	2.15	2.03E-04	1.67	8.03E-06	2.54	8.81E-06	2.53
	$h/8$	9.84E-06	2.10	6.54E-05	1.63	1.39E-06	2.53	1.53E-06	2.53
	$h/16$	2.35E-06	2.07	2.15E-05	1.60	2.41E-07	2.53	2.67E-07	2.52
		$\|e_{\boldsymbol{\sigma}}\|_{L^1(\Omega)}$		$\|e_{\boldsymbol{q}}\|_{L^1(\Omega)}$		$\|e_u\|_{L^1(\Omega)}$		$\|e_{u^*}\|_{L^1(\Omega)}$	
k	Mesh	Error	Order	Error	Order	Error	Order	Error	Order
0	h	3.94E-02	-	4.02E-02	-	6.34E-02	-	6.34E-02	-
	$h/2$	2.26E-02	0.80	2.52E-02	0.67	3.12E-02	1.02	3.12E-02	1.02
	$h/4$	1.19E-02	0.93	1.39E-02	0.86	1.55E-02	1.01	1.55E-02	1.01
	$h/8$	6.11E-03	0.96	7.30E-03	0.93	7.75E-03	1.00	7.75E-03	1.00
	$h/16$	3.10E-03	0.98	3.74E-03	0.96	3.87E-03	1.00	3.87E-03	1.00
1	h	9.94E-03	-	1.25E-02	-	2.04E-03	-	1.76E-03	-
	$h/2$	2.57E-03	1.95	3.50E-03	1.84	3.94E-04	2.37	2.89E-04	2.61
	$h/4$	6.46E-04	1.99	9.13E-04	1.94	7.60E-05	2.37	4.22E-05	2.77
	$h/8$	1.69E-04	2.01	2.30E-04	1.99	1.62E-05	2.23	5.78E-06	2.87
	$h/16$	3.98E-05	2.00	5.74E-05	2.00	3.83E-06	2.08	7.61E-07	2.93
2	h	6.39E-04	-	9.15E-04	-	1.88E-04	-	1.29E-04	-
	$h/2$	1.04E-04	2.61	1.83E-05	2.32	2.13E-05	3.14	1.50E-05	3.10
	$h/4$	1.57E-05	2.74	3.30E-06	2.47	2.53E-06	3.08	1.68E-06	3.16
	$h/8$	2.24E-06	2.81	5.69E-07	2.54	3.03E-07	3.06	1.87E-07	3.17
	$h/16$	3.05E-07	2.87	9.64E-08	2.56	3.65E-08	3.05	2.07E-08	3.17

A similar assessment can be made of Algorithm II and the preconditioned Algorithm II; see Table 7. In conclusion, the preconditioners greatly improve the performance of both Algorithm I and Algorithm II.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

TABLE 6

Performance of Algorithm I *with and without preconditioning on the smooth test with* $p = 3$, $\epsilon = 0$, *and* $h = 0.707$. *The line search parameters are* $c_1 = 0.6$, $c_2 = 10^{-4}$, $\alpha^0 = 1$, *and* $\delta_l = 10^{-4}$.

			Algorithm I		Preconditioned Algorithm I				
k	Mesh	N_{dof}	N_{iter}	CPU	N_{iter}	CPU	CPU_{prec}	CPU_{line}	CPU_{local}
0	h	8	20	1.12	4	0.17	0.13	0.02	7.00%
	$h/2$	40	49	3.87	4	0.56	0.39	0.07	16.56%
	$h/4$	176	127	32.27	5	2.76	1.80	0.26	27.03%
1	h	16	39	8.28	5	3.03	0.57	0.29	51.49%
	$h/2$	80	115	89.34	5	9.62	2.12	1.14	63.62%
	$h/4$	352	203	622.82	5	35.74	8.28	4.44	67.54%
2	h	24	50	59.83	5	11.99	2.06	1.49	72.06%
	$h/2$	120	117	475.58	5	45.68	7.94	6.30	76.51%
	$h/4$	528	477	6879.89	5	176.32	32.57	24.57	77.15%
3	h	32	69	361.01	5	45.41	6.49	6.71	80.60%
	$h/2$	160	156	3187.20	5	216.19	30.38	31.60	82.51%
	$h/4$	704	565	35548.47	5	709.64	108.72	101.85	80.83%

TABLE 7

Performance of Algorithm II *with and without preconditioning on the smooth test with* $p = 3$, $\epsilon = 0$, *and* $h = 0.707$. *The line search parameters are* $c_1 = 0.6$, $c_2 = 10^{-4}$, $\alpha^0 = 1$, *and* $\delta_l = 10^{-4}$.

			Algorithm II		Preconditioned Algorithm II				
k	Mesh	N_{dof}	N_{iter}	CPU	N_{iter}	CPU	CPU_{prec}	CPU_{line}	CPU_{local}
0	h	16	48	1.23	8	1.06	0.31	0.02	4.91%
	$h/2$	72	56	2.35	10	2.35	1.24	0.04	10.21%
	$h/4$	304	173	19.23	10	7.22	4.77	0.12	13.14%
1	h	40	224	9.46	10	2.80	1.52	0.05	14.64%
	$h/2$	176	772	116.95	10	8.57	5.67	0.16	19.72%
	$h/4$	736	2664	1583.38	10	32.04	22.42	0.61	21.10%
2	h	72	569	93.49	10	8.46	5.46	0.17	22.46%
	$h/2$	312	2075	1324.50	10	30.76	20.94	0.64	24.41%
	$h/4$	1296	8301	21079.33	10	120.54	83.36	2.55	25.86%
3	h	112	1525	809.10	10	23.90	15.69	0.53	26.19%
	$h/2$	480	9255	19440.21	10	92.78	62.56	2.10	27.05%
	$h/4$	1984	38821	326093.19	10	368.71	250.14	8.40	27.25%

4.3.2. Preconditioned Algorithm I and preconditioned Algorithm II. Here, we compare the performance of the preconditioned Algorithms I and II.

Convergence speed. In Table 6 (preconditioned Algorithm I) and Table 7 (preconditioned Algorithm II), we observe that even though the preconditioned Algorithm I needs fewer iterations, it is usually slower than the preconditioned Algorithm II. This behavior is mainly caused by the slow convergence of the local nonlinear solver. The same reason leads to a much more expensive line search procedure for preconditioned Algorithm I, which is accounted for by the column CPU_{line}. On the other hand, the column CPU_{prec} indicates that the preconditioning procedure for Algorithm II is more expensive. The main reason is that preconditioning for either algorithm consists of solving a linear HDG system in each global iteration. Therefore CPU_{prec} predominately depends on the total number of global iterations. Since the number N_{iter} of the preconditioned Algorithm II is roughly twice that of preconditioned Algorithm I, the CPU_{prec} of the preconditioned Algorithm II is also roughly twice that of the preconditioned Algorithm I. However, it is actually a little bit more than twice since the preconditioning of Algorithm II requires assembling each component of the preconditioned gradient. Finally, the column CPU_{local} in Table 6 shows that the total amount of time for computing the local solver for the preconditioned Algorithm I can take up

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

more than 80% of the total CPU time. In contrast, since we have only linear local problems for the preconditioned Algorithm II, the local solver computation takes up only around 25% of the total CPU time. Note that the high values of CPU_{local} also indicate that the preconditioned Algorithm I could benefit from parallelization.

Number of unknowns. In Tables 6 and 7, we compare the number of unknowns needed to evaluate the functionals used by Algorithms I and II. We can compute those numbers exactly for a uniform mesh with $2m^2$ triangles and for polynomials of degree k:

$$N_{dof}^{\mathrm{I}} = (3m^2 - 2m)(k+1), \qquad N_{dof}^{\mathrm{II}} = ((k+5)m^2 - 2m)(k+1).$$

So, when as m increases, the ratio $N_{dof}^{\mathrm{II}}/N_{dof}^{\mathrm{I}}$ becomes closer to $(k+5)/3$. Algorithm II thus needs much more memory than Algorithm I.

Robustness with respect to variations in p. Our results suggest that preconditioned Algorithm II is more robust than preconditioned Algorithm I when p is of a wide range. For instance, consider a simple test problem where f and g satisfy the exact solution $u(x,y) = e^{x+y}$ on the domain $\Omega = (0,1)^2$. For $p = 1$, we observe that even for a smooth solution, preconditioned Algorithm I does *not* converge when using polynomials of degree $k \geq 1$. However, preconditioned Algorithm II remains convergent. In fact, for the more difficult test problems such as (1.3) with large p, only preconditioned Algorithm II remains convergent. Therefore, when considering a problem without any knowledge of exact solution, we should use only the preconditioned Algorithm II.

4.4. More difficult problems. In this section, we explore the ability of the HDG method to capture the solutions displaying large or small gradients; we use the preconditioned Algorithm II with stabilization function $\tau = 1$.

4.4.1. A problem with cone-shaped solution. Let us use a *modification* of the classical test problem (1.3) to explore the performance of our HDG method on a cone-shaped solution. We discretize the unit square domain $\Omega = (0,1) \times (0,1)$ into uniform triangles and translate the exact solution (1.3) so that the tip of its cone lies *inside* the domain at $(0.5, 0.5)$:

$$u(x,y) = \frac{p-1}{p}\left(\frac{1}{2}\right)^{\frac{1}{p-1}} (1 - ((x-0.5)^2 + (y-0.5)^2))^{\frac{p}{2(p-1)}}). \tag{4.3}$$

We use Algorithm 3 to generate the initial guess and take the line search parameters to be $c_1 = 0.9, c_2 = 10^{-4}$, and $\epsilon = 10^{-2}$. In Figure 1, we present a cut-view comparison along the line $y = x$ when $p = 50$ near and away from the tip of the cone. We can observe that away from the tip, the postprocessing provides better approximations, while near the tip the quality of postprocessed approximations degrades due to the poor regularity, as expected. However, using $k = 2$ provides better approximations than using $k = 1$ both near and away from the tip. In Figure 2, we present details around the tip of the cone of the approximation and its postprocessing. We see an increased size of the jumps *typical* of DG methods when the exact solution is difficult to approximate. We can also observe that the $k = 2$ approximation is much better than the $k = 1$ approximation even near the tip. Moreover, the $k = 2$ postprocessing is also slightly better than the piecewise quadratic postprocessing. These observations are in favor of considering higher order approximation and postprocessing.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

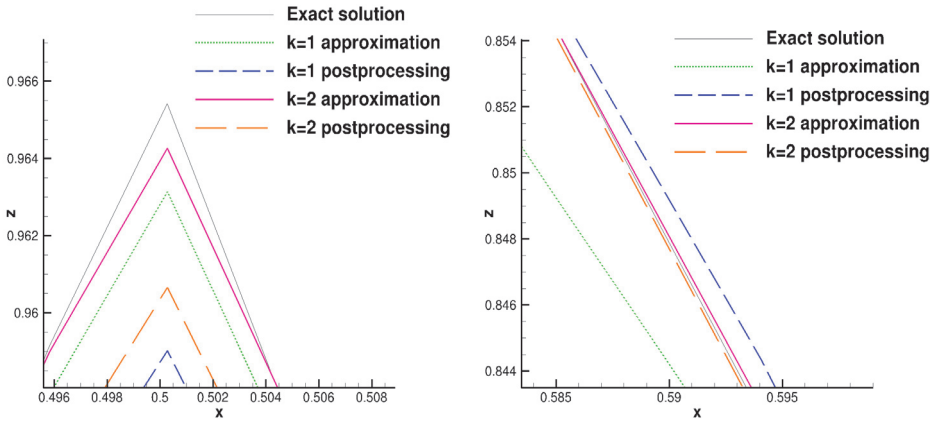


FIG. 1. *Cut-view comparison on test problem* (4.3) *with* $p = 50$ *along the line* $y = x$*: closeup view near the kink (left) and closeup view away from the kink (right).*

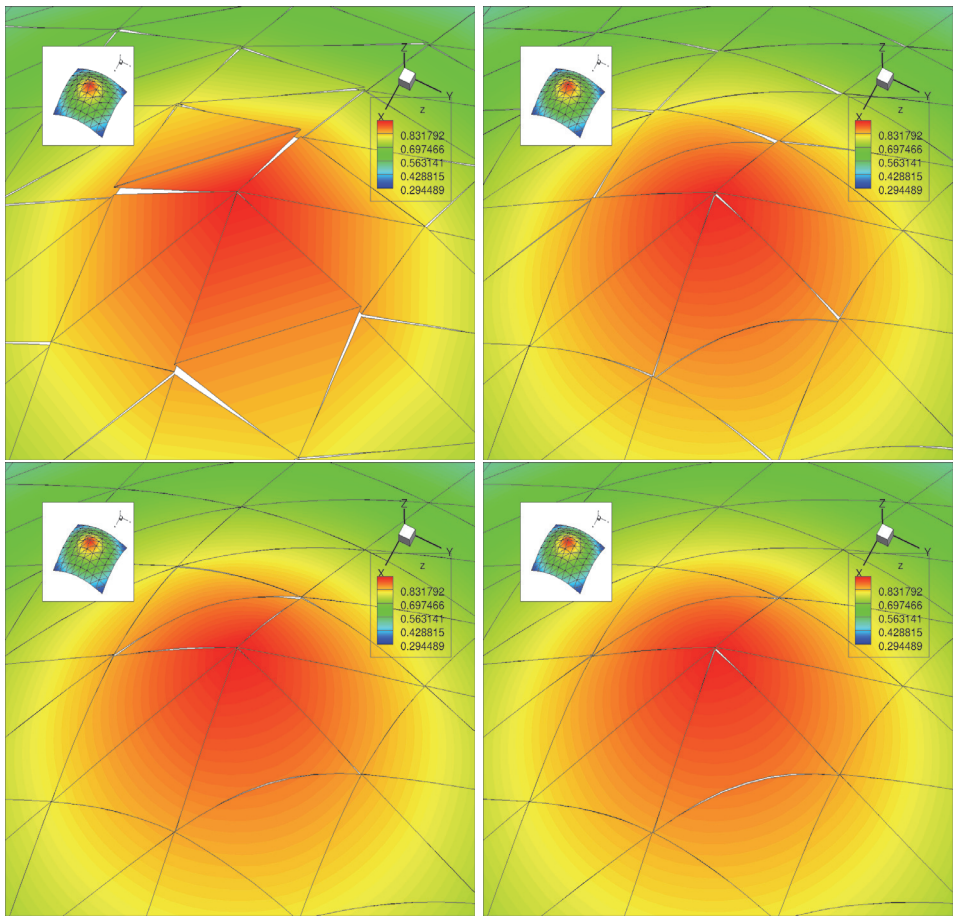


FIG. 2. *Closeup comparison between piecewise-linear approximation (top left), its piecewise quadratic postprocessing (top right), piecewise quadratic approximation (bottom left), its piecewise cubic postprocessing (bottom right) on test problem* (4.3) *with* $p = 50$.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

4.4.2. A problem from torsional creep. Here, we explore the performance of our HDG method on the following Dirichlet problems model torsional creep [25]:

$$-\nabla \cdot (|\boldsymbol{\nabla} u|^{p-2}\boldsymbol{\nabla} u) = f \text{ in } \Omega = (0,1)^2, \ u = 0 \text{ on } \partial\Omega. \tag{4.4}$$

These solutions exhibit large gradients. Indeed, in [26, 4], the authors show that $\lim_{p\to\infty} u = d(x, \partial\Omega)$ (d is a distance function) in the region at which the support of f covers the set of points where the gradient $\boldsymbol{\nabla} d(\cdot, \Omega)$ is discontinuous. They also show that, as $p \to 1$, and the size of Ω is suitably chosen, the solution tends to a function exhibiting discontinuities.

In our numerical experiments, we take

$$f(x,y) = \begin{cases} 8 & \text{if } (x,y) \in [0.25, 0.75]^2, \\ 0 & \text{otherwise,} \end{cases}$$

so that the first behavior is expected to hold in the region where $f > 0$, namely, within the set $[0.25, 0.75]^2$. Therefore we expect to observe a forming "pyramid" in that region. Furthermore, to observe the second behavior, we scale the source term instead of varying the size of domain. By setting the magnitude of f to be 8 inside $[0.25, 0.75]^2$, we can detect that the approximate solution tends to be discontinuous as $p \to 1$.

We use Algorithm 3 and take the backtrack line search parameters to be $c_1 = 0.9$, $c_2 = 10^{-4}$, $\delta = 10^{-8}$, and $\epsilon = 10^{-4}$. We display the numerical results for piecewise-constant approximations in Figure 3, for piecewise-linear approximations in Figure 4, and for piecewise-quadratic approximations in Figure 5. In each Figure, we display the approximation error for both u_h and its postprocessing u_h^*; we take $p = 1.1, 2$, and 11. We can observe that when $p = 1.1$, the solution presents rather large gradients. Again, as is typical of DG approximations, the solution u_h displays significant interelement jumps near the region of large gradients, especially for the piecewise-linear approximation, while the quadratic postprocessing u_h^* successfully smoothes them out, as expected. On the other hand, we can observe that when p equals the dual of 1.1, namely, 11, the solution tends to exhibit the shape of the distance function within $[1/4, 3/4]^2$. The approximation u_h then becomes "rough" near the tip of the forming "pyramid" and we see the expected increase in the interelement jumps near it; its postprocessing is still able to provide a better approximation away from the tip.

5. Concluding remarks. In this paper, we introduced the first HDG method for the p-Laplacian equation. We gave ample numerical evidence that the method displays the same convergence properties as in the linear case and that it provides good approximations even for nonsmooth solutions. To solve the difficult nonlinear equations, we introduced two nonlinear gradient descent algorithms based on two characterizations of the HDG methods as minimization problem. Frozen coefficient preconditioners, which demonstrated h- and k-independent asymptotic behavior, were used. We showed that Algorithm II is better than Algorithm I in terms of convergence speed and robustness with respect to p, even though it uses more unknowns.

The extension of our approach to nonlinear elasticity is the subject of ongoing work.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

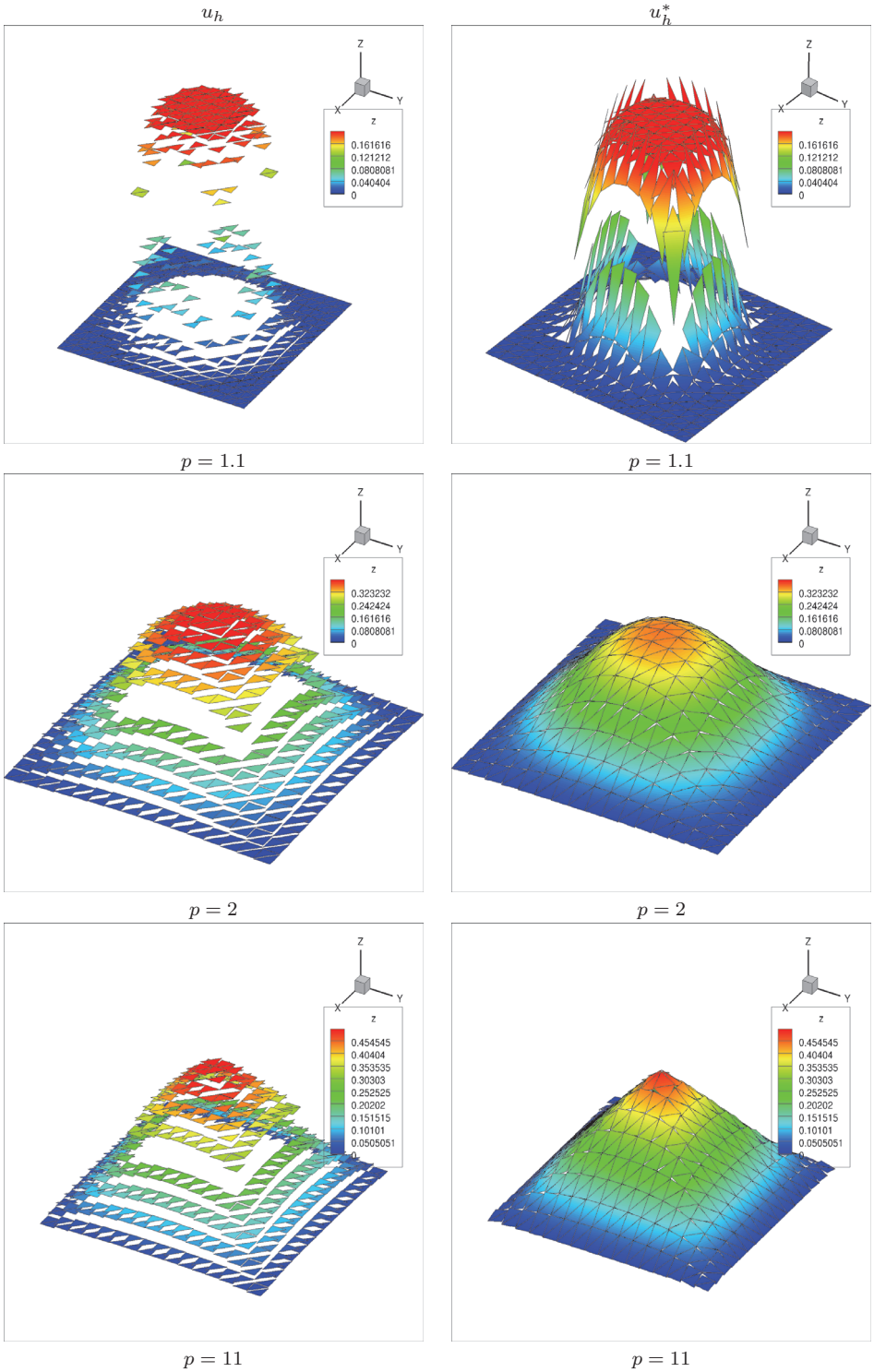


FIG. 3. *The piecewise-constant approximation* u_h *and* u_h^* *for* $f = 8 \cdot \chi_{[1/4,3/4]^2}$ *with various* p *for test problem* (4.4).

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

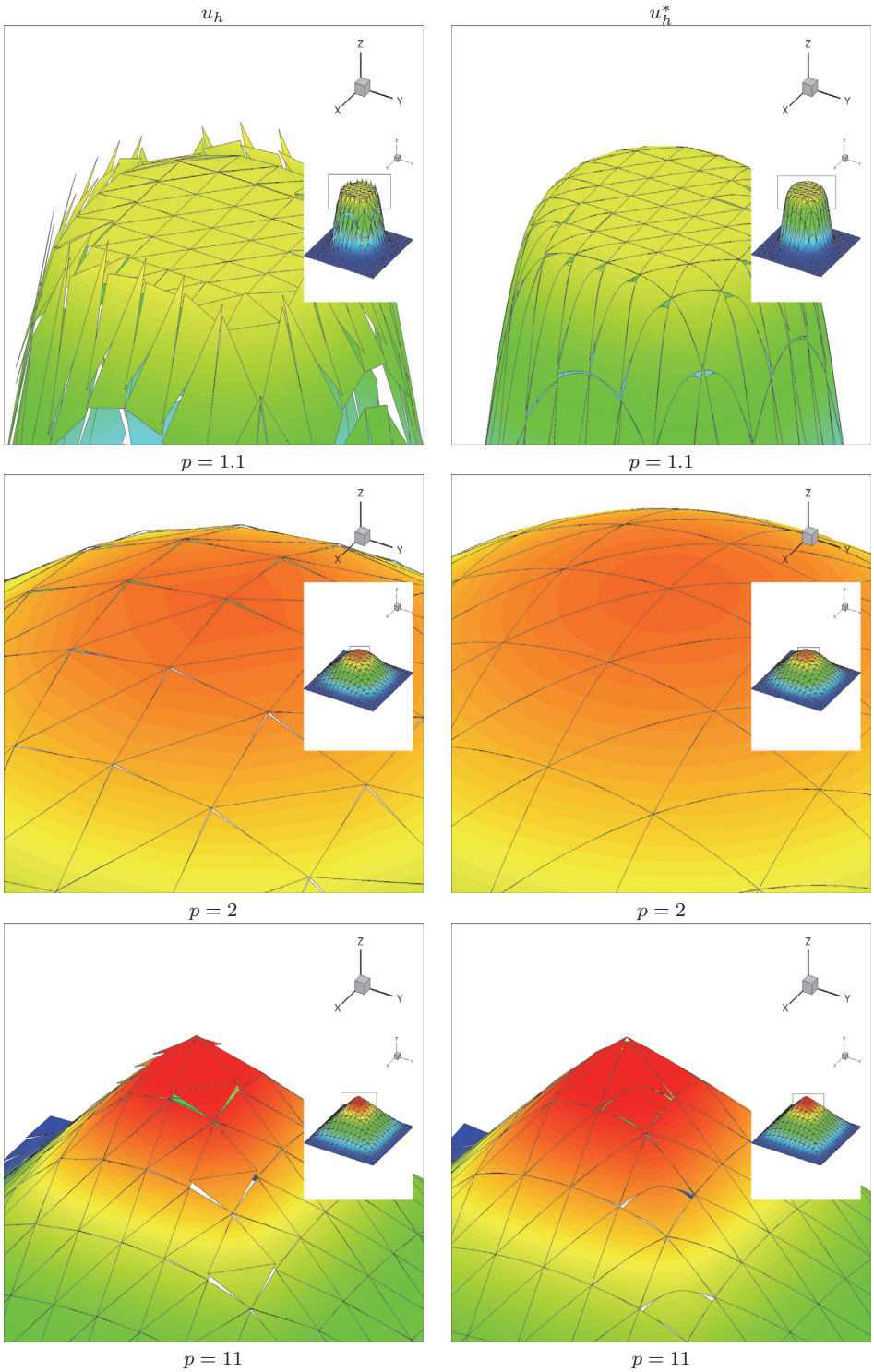


FIG. 4. *The closeup comparison of piecewise-linear approximation u_h and the corresponding postprocessed solution u_h^* for various p for test problem* (4.4).

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

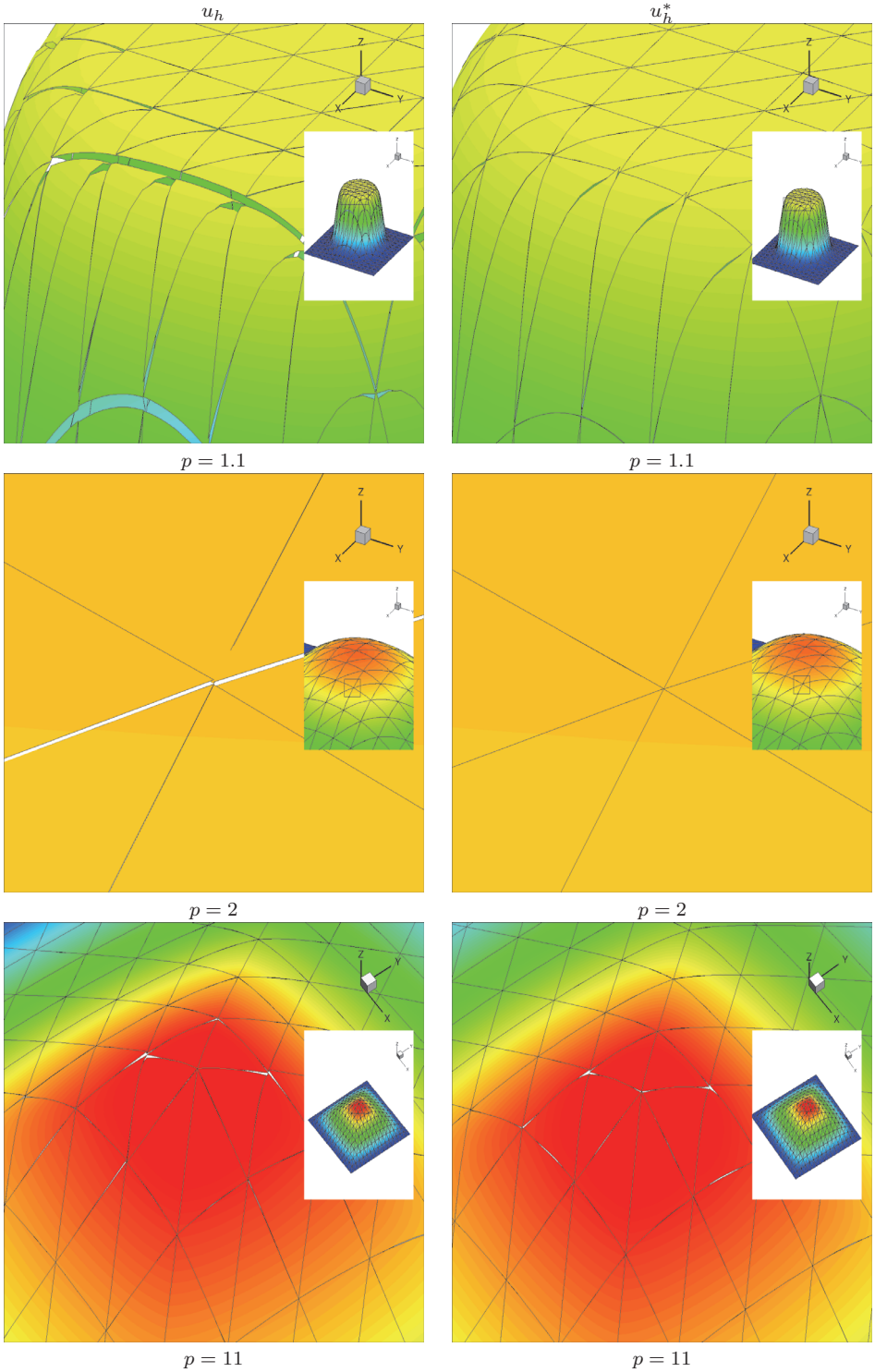


FIG. 5. *The closeup comparison of piecewise-quadratic approximation u_h and the corresponding postprocessed solution u_h^* for various p for test problem* (4.4).

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

REFERENCES

[1] M. Ainsworth and D. Kay, *The approximation theory for the p-version finite element method and application to non-linear elliptic PDEs*, Numer. Math., 82 (1999), pp. 351–388.

[2] D. N. Arnold and F. Brezzi, *Mixed and nonconforming finite element methods: Implementation, postprocessing and error estimates*, RAIRO Modél. Math. Anal. Numér., 19 (1985), pp. 7–32.

[3] J. W. Barrett and W. B. Liu, *Finite element approximation of the p-Laplacian*, Math. Comp., 61 (1993), pp. 523–537.

[4] T. Bhattacharya, E. DiBenedetto, and J. Manfredi, *Limits as $p \to \infty$ of $\Delta_p u_p = f$ and related extremal problems*, Rend. Semin. Mat. Univ. Politec. Torino (1989), pp. 15–68.

[5] G. Bognár, *On similarity solution of a boundary layer problem for power-law fluids*, Math. Bohem., 137 (2012), pp. 139–148.

[6] G. Bouchitté, G. Buttazzo, and L. De Pascale, *A p-Laplacian approximation for some mass optimization problems*, J. Optim. Theory Appl., 118 (2003), pp. 1–25.

[7] A. Buffa and C. Ortner, *Compact embeddings of broken Sobolev spaces and applications*, IMA J. Numer. Anal., 29 (2009), pp. 827–855.

[8] E. Burman and A. Ern, *Discontinuous Galerkin approximation with discrete variational principle for the nonlinear Laplacian*, C. R. Math. Acad. Sci. Paris, 346 (2008), pp. 1013–1016.

[9] P. G. Ciarlet, *The Finite Element Method for Elliptic Problems*, Stud. Math. Appl. 4, North-Holland, Amsterdam, 1978.

[10] B. Cockburn, J. Gopalakrishnan, and R. Lazarov, *Unified hybridization of discontinuous Galerkin, mixed, and continuous Galerkin methods for second order elliptic problems*, SIAM J. Numer. Anal., 47 (2009), pp. 1319–1365.

[11] B. Cockburn, J. Gopalakrishnan, and F. J. Sayas, *A projection-based error analysis of HDG methods*, Math. Comp., 79 (2010), pp. 1351–1367.

[12] B. Cockburn, J. Guzmán, and H. Wang, *Superconvergent discontinuous Galerkin methods for second-order elliptic problems*, Math. Comp., 78 (2009), pp. 1–24.

[13] B. Cockburn and W. Zhang, *A posteriori error analysis for hybridizable discontinuous Galerkin methods for second order elliptic problems*, SIAM J. Numer. Anal., 51 (2013), pp. 676–693.

[14] E. Creuse, M. Farhloul, and L. Paquet, *A posteriori error estimation for the dual mixed finite element method for the p-Laplacian in a polygonal domain*, Comput. Methods Appl. Mech. Engrg., 196 (2007), pp. 2570–2582.

[15] E. DiBenedetto and M. A. Herrero, *Nonnegative solutions of the evolution p-Laplacian equation. Initial traces and Cauchy problem when $1 < p < 2$*, Arch. Ration. Mech. Anal., 111 (1990), pp. 225–290.

[16] L. Diening, D. Köner, M. Růžička, and I. Toulopoulos, *A local discontinuous Galerkin approximation for systems with p-structure*, IMA J. Numer. Anal., 34 (2014), pp. 1447–1488.

[17] I. Faragó and J. Karátson, *Numerical solution of nonlinear elliptic problems via preconditioning operators: Theory and applications*, in Advances in Computation: Theory and Practice 11, Nova Science Publishers, Hauppauge, NY, 2002.

[18] M. Farhloul, *A mixed finite element method for a nonlinear Dirichlet problem*, IMA J. Numer. Anal., 18 (1998), pp. 121–132.

[19] M. Farhloul and H. Manouzi, *On a mixed finite element method for the p-Laplacian*, Can. Appl. Math. Q., 8 (2000), pp. 67–78.

[20] R. Glowinski and A. Marrocco, *Sur l'approximation, par éléments finis d'ordre un, et la résolution, par pénalisation-dualité, d'une classe de problèmes de Dirichlet non lineaires*, RAIRO Anal. Numér., 9 (1975), pp. 41–76.

[21] W. Hager and H. Zhang, *A survey of nonlinear conjugate gradient methods*, Pac. J. Optim., 2 (2006), pp. 35–58.

[22] A. Hirn, *Approximation of the p-Stokes equations with equal-order finite elements*, J. Math. Fluid Mech., 15 (2013), pp. 65–88.

[23] Y. Q. Huang, R. Li, and W. B. Liu, *Preconditioned descent algorithms for p-Laplacian*, J. Sci. Comput., 32 (2007), pp. 343–371.

[24] H. Kabaria, A. J. Lew, and B. Cockburn, *A hybridizable discontinuous Galerkin formulation for non-linear elasticity*, Comput. Methods Appl. Mech. Engrg., 283 (2015), pp. 303–329.

[25] B. Kawohl, *On a family of torsional creep problems*, J. Reine Angew. Math., 410 (1990), pp. 1–22.

[26] B. Kawohl, *Variations on the p-Laplacian*, in Nonlinear Elliptic Partial Differential Equations, Contemp. Math. 540, AMS, Providence, RI, 2011, pp. 35–46.

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.

[27] O. A. Ladyženskaja, *New equations for the description of the motions of viscous incompressible fluids, and global solvability for their boundary value problems*, Tr. Mat. Inst. Steklova, 102 (1967), pp. 85–104.
[28] M.-C. Pélissier and L. Reynaud, *Étude d'un modèle mathématique d'écoulement de glacier*, C. R. Acad. Sci. Paris Sér. A, 279 (1974), pp. 531–534.
[29] Y. Peres and S. Sheffield, *Tug-of-war with noise: A game-theoretic view of the p-Laplacian*, Duke Math. J., 145 (2008), pp. 91–120.
[30] J. R. Philip, *n-diffusion*, Austral. J. Phys., 14 (1961), pp. 1–13.
[31] Z. Shi and J. Shen, *Convergence of descent method without line search*, Appl. Math. Comput., 167 (2005), pp. 94–107.
[32] R. E. Showalter and N. J. Walkington, *Diffusion of fluid in a fissured medium with microstructure*, SIAM J. Math. Anal., 22 (1991), pp. 1702–1722.
[33] G. Zhou and C. Feng, *The steepest descent algorithm without line search for p-Laplacian*, Appl. Math. Comput., 224 (2013), pp. 36–45.

Downloaded 03/15/22 to 222.195.77.3 . Redistribution subject to SIAM license or copyright; see https://epubs.siam.org/terms-privacy

Copyright © by SIAM. Unauthorized reproduction of this article is prohibited.